

Отчёт по лабораторной работе №4

Компьютерный практикум по статистическому анализу данных

Линейная алгебра

Выполнил: Исаев Булат Абубакарович,
НПИбд-01-22, 1132227131

Содержание

1	Цель работы	1
2	Выполнение лабораторной работы.....	1
2.1	Поэлементные операции над многомерными массивами	1
2.2	Транспонирование, след, ранг, определитель и инверсия матрицы	3
2.3	Вычисление нормы векторов и матриц, повороты, вращения	5
2.4	Матричное умножение, единичная матрица, скалярное произведение	7
2.5	Факторизация. Специальные матричные структуры	8
2.6	Общая линейная алгебра.....	17
2.7	Самостоятельная работа.....	18
3	Вывод.....	25
4	Список литературы. Библиография	26

1 Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

2 Выполнение лабораторной работы

2.1 Поэлементные операции над многомерными массивами

Для матрицы 4×3 рассмотрим поэлементные операции сложения и произведения её элементов (рис. 1):

```
[1]: # Массив 4x3 со случайными целыми числами (от 1 до 20):  
a = rand(1:20,(4,3))
```

```
[1]: 4x3 Matrix{Int64}:  
 3 16 19  
12 1 17  
17 2 13  
20 1 10
```

```
[2]: # Поэлементная сумма:  
sum(a)
```

```
[2]: 131
```

```
[3]: # Поэлементная сумма по столбцам:  
sum(a,dims=1)
```

```
[3]: 1x3 Matrix{Int64}:  
52 20 59
```

```
[4]: # Поэлементная сумма по строкам:  
sum(a,dims=2)
```

```
[4]: 4x1 Matrix{Int64}:  
38  
30  
32  
31
```

```
[5]: # Поэлементное произведение:  
prod(a)
```

```
[5]: 16446643200
```

```
[6]: # Поэлементное произведение по столбцам:  
prod(a,dims=1)
```

```
[6]: 1x3 Matrix{Int64}:  
12240 32 41990
```

```
[7]: # Поэлементное произведение по строкам:  
prod(a,dims=2)
```

```
[7]: 4x1 Matrix{Int64}:  
912  
204  
442  
200
```

Рис. 1: Поэлементные операции сложения и произведения элементов матрицы

Для работы со средними значениями можно воспользоваться возможностями пакета Statistics (рис. 2):

```
[14]: using Statistics
      # Вычисление среднего значения массива:
      mean(a)
```

```
[14]: 10.916666666666666
```

```
[15]: # Среднее по столбцам:
      mean(a,dims=1)
```

```
[15]: 1×3 Matrix{Float64}:
      13.0  5.0  14.75
```

```
[16]: # Среднее по строкам:
      mean(a,dims=2)
```

```
[16]: 4×1 Matrix{Float64}:
      12.666666666666666
      10.0
      10.666666666666666
      10.333333333333334
```

Рис. 2: Использование возможностей пакета *Statistics* для работы со средними значениями

2.2 Транспонирование, след, ранг, определитель и инверсия матрицы

Для выполнения таких операций над матрицами, как транспонирование, диагонализация, определение следа, ранга, определителя матрицы и т.п. можно воспользоваться библиотекой (пакетом) *LinearAlgebra* (рис. 3 - рис. 4):

```
[17]: # Подключение пакета LinearAlgebra:  
using LinearAlgebra  
  
# Массив 4x4 со случайными целыми числами (от 1 до 20):  
b = rand(1:20,(4,4))
```

```
[17]: 4x4 Matrix{Int64}:  
 11  2 15 18  
 14  1 11  5  
 16 13 20  2  
 16 15 12 20
```

```
[18]: # Транспонирование:  
transpose(b)
```

```
[18]: 4x4 transpose(::Matrix{Int64}) with eltype Int64:  
 11 14 16 16  
  2  1 13 15  
 15 11 20 12  
 18  5  2 20
```

```
[19]: # След матрицы (сумма диагональных элементов):  
tr(b)
```

```
[19]: 52
```

```
[20]: # Извлечение диагональных элементов как массив:  
diag(b)
```

```
[20]: 4-element Vector{Int64}:  
 11  
  1  
 20  
 20
```

Рис. 3: Использование библиотеки *LinearAlgebra* для выполнения определённых операций

```
[21]: # Ранг матрицы:  
rank(b)
```

```
[21]: 4
```

```
[22]: # Инверсия матрицы (определение обратной матрицы):  
inv(b)
```

```
[22]: 4x4 Matrix{Float64}:  
-0.0727558  0.133917  -0.0407321  0.0360741  
-0.0368589  -0.0629842  0.0350362  0.0454154  
0.0782745  -0.0640474  0.0629588  -0.0607311  
0.0388841  -0.0214673  -0.0314668  0.0235178
```

```
[23]: # Определитель матрицы:  
det(b)
```

```
[23]: 39502.00000000001
```

```
[24]: # Псевдообратная функция для прямоугольных матриц:  
pinv(a)
```

```
[24]: 3x4 Matrix{Float64}:  
-0.00364399  -0.0494537  0.0239468  0.0598641  
0.0654338   -0.105971  0.00457614  0.049878  
-0.0020399   0.0963647  -0.00540303  -0.0529203
```

Рис. 4: Использование библиотеки *LinearAlgebra* для выполнения определённых операций

2.3 Вычисление нормы векторов и матриц, повороты, вращения

Для вычисления нормы используется `LinearAlgebra.norm(x)` (рис. 5):

```

[25]: # Создание вектора X:
      X = [2, 4, -5]

[25]: 3-element Vector{Int64}:
       2
       4
      -5

[26]: # Вычисление евклидовой нормы:
      norm(X)

[26]: 6.708203932499369

[27]: # Вычисление p-нормы:
      p = 1
      norm(X,p)

[27]: 11.0

[28]: # Расстояние между двумя векторами X и Y:
      X = [2, 4, -5];
      Y = [1, -1, 3];
      norm(X-Y)

[28]: 9.486832980505138

[29]: # Проверка по базовому определению:
      sqrt(sum((X-Y).^2))

[29]: 9.486832980505138

[30]: # Угол между двумя векторами:
      acos((transpose(X)*Y)/(norm(X)*norm(Y)))

[30]: 2.4404307889469252

```

Рис. 5: Использование *LinearAlgebra.norm(x)*

Вычислим нормы для двумерной матрицы (рис. 6):

```
[31]: # Создание матрицы:  
d = [5 -4 2 ; -1 2 3; -2 1 0]
```

```
[31]: 3x3 Matrix{Int64}:  
  5  -4  2  
 -1   2  3  
 -2   1  0
```

```
[32]: # Вычисление Евклидовой нормы:  
opnorm(d)
```

```
[32]: 7.147682841795258
```

```
[33]: # Вычисление p-нормы:  
p=1  
opnorm(d,p)
```

```
[33]: 8.0
```

```
[34]: # Поворот на 180 градусов:  
rot180(d)
```

```
[34]: 3x3 Matrix{Int64}:  
  0   1  -2  
  3   2  -1  
  2  -4   5
```

```
[35]: # Переворачивание строк:  
reverse(d,dims=1)
```

```
[35]: 3x3 Matrix{Int64}:  
 -2   1   0  
 -1   2   3  
  5  -4   2
```

```
[36]: # Переворачивание столбцов  
reverse(d,dims=2)
```

```
[36]: 3x3 Matrix{Int64}:  
  2  -4   5  
  3   2  -1  
  0   1  -2
```

Рис. 6: Вычисление нормы для двумерной матрицы

2.4 Матричное умножение, единичная матрица, скалярное произведение

Выполним примеры матричного умножения, единичной матрицы и скалярного произведения (рис. 7):

```
[37]: # Матрица 2x3 со случайными целыми значениями от 1 до 10:  
A = rand(1:10,(2,3))
```

```
[37]: 2x3 Matrix{Int64}:  
 9  1  2  
10  6  5
```

```
[38]: # Матрица 3x4 со случайными целыми значениями от 1 до 10:  
B = rand(1:10,(3,4))
```

```
[38]: 3x4 Matrix{Int64}:  
 9  3  1  4  
 6  9  5  4  
 6  4 10  3
```

```
[39]: # Произведение матриц A и B:  
A*B
```

```
[39]: 2x4 Matrix{Int64}:  
99  44  34  46  
156 104  90  79
```

```
[40]: # Единичная матрица 3x3:  
Matrix{Int}(I, 3, 3)
```

```
[40]: 3x3 Matrix{Int64}:  
 1  0  0  
 0  1  0  
 0  0  1
```

```
[41]: # Скалярное произведение векторов X и Y:  
X = [2, 4, -5]  
Y = [1, -1, 3]  
dot(X,Y)
```

```
[41]: -17
```

```
[42]: # тоже скалярное произведение:  
X*Y
```

```
[42]: -17
```

Рис. 7: Примеры матричного умножения, единичной матрицы и скалярного произведения

2.5 Факторизация. Специальные матричные структуры

Рассмотрим несколько примеров. Для работы со специальными матричными структурами потребуется пакет LinearAlgebra.

Решение систем линейных алгебраических уравнений $Ax = b$ (рис. 8):


```
[43]: # Задаём квадратную матрицу 3x3 со случайными значениями:  
A = rand(3, 3)
```

```
[43]: 3×3 Matrix{Float64}:  
 0.326169  0.935216  0.282772  
 0.448333  0.427768  0.0281235  
 0.285051  0.531141  0.774524
```

```
[44]: # Задаём единичный вектор:  
x = fill(1.0, 3)
```

```
[44]: 3-element Vector{Float64}:  
 1.0  
 1.0  
 1.0
```

```
[45]: # Задаём вектор b:  
b = A*x
```

```
[45]: 3-element Vector{Float64}:  
 1.5441576124622873  
 0.9042245678093209  
 1.5907152346338276
```

```
[46]: # Решение исходного уравнения получаем с помощью функции \  
# (убеждаемся, что x - единичный вектор):  
A\b
```

```
[46]: 3-element Vector{Float64}:  
 1.0  
 1.0  
 1.0
```

Рис. 8: Решение систем линейный алгебраических уравнений $AX = B$

Julia позволяет вычислять LU-факторизацию и определяет составной тип факторизации для его хранения (рис. 9):

```
[47]: # LU-факторизация:
      Alu = lu(A)

[47]: LU{Float64, Matrix{Float64}, Vector{Int64}}
      L factor:
      3×3 Matrix{Float64}:
        1.0      0.0      0.0
        0.727515 1.0      0.0
        0.635801 0.415323 1.0
      U factor:
      3×3 Matrix{Float64}:
        0.448333 0.427768 0.0281235
        0.0      0.624009 0.262312
        0.0      0.0      0.647699
```

```
[48]: # Матрица перестановок:
      Alu.P
```

```
[48]: 3×3 Matrix{Float64}:
      0.0 1.0 0.0
      1.0 0.0 0.0
      0.0 0.0 1.0
```

```
[49]: # Вектор перестановок:
      Alu.p
```

```
[49]: 3-element Vector{Int64}:
      2
      1
      3
```

```
[50]: # Матрица L:
      Alu.L
```

```
[50]: 3×3 Matrix{Float64}:
      1.0      0.0      0.0
      0.727515 1.0      0.0
      0.635801 0.415323 1.0
```

```
[51]: # Матрица U:
      Alu.U
```

```
[51]: 3×3 Matrix{Float64}:
      0.448333 0.427768 0.0281235
      0.0      0.624009 0.262312
      0.0      0.0      0.647699
```

Рис. 9: Пример вычисления LU-факторизации и определение составного типа факторизации для его хранения

Исходная система уравнений $Ax = b$ может быть решена или с использованием исходной матрицы, или с использованием объекта факторизации (рис. 10):

```
[52]: # Решение СЛАУ через матрицу A:  
A\b
```

```
[52]: 3-element Vector{Float64}:  
 1.0  
 1.0  
 1.0
```

```
[53]: # Решение СЛАУ через объект факторизации:  
Alu\b
```

```
[53]: 3-element Vector{Float64}:  
 1.0  
 1.0  
 1.0
```

```
[54]: # Детерминант матрицы A:  
det(A)
```

```
[54]: -0.18120261450866698
```

```
[55]: # Детерминант матрицы A через объект факторизации:  
det(Alu)
```

```
[55]: -0.18120261450866698
```

Рис. 10: Пример решения с использованием исходной матрицы и с использованием объекта факторизации

Julia позволяет вычислять QR-факторизацию и определяет составной тип факторизации для его хранения (рис. 11):

```

[56]: # QR-факторизация:
Aqr = qr(A)

[56]: LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}
Q factor: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
R factor:
3×3 Matrix{Float64}:
-0.623412 -1.0398 -0.522317
 0.0      0.508472 0.284696
 0.0      0.0      0.571639

[57]: # Матрица Q:
Aqr.Q

[57]: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}

[58]: # Матрица R:
Aqr.R

[58]: 3×3 Matrix{Float64}:
-0.623412 -1.0398 -0.522317
 0.0      0.508472 0.284696
 0.0      0.0      0.571639

[59]: # Проверка, что матрица Q - ортогональная:
Aqr.Q'*Aqr.Q

[59]: 3×3 Matrix{Float64}:
 1.0      0.0      0.0
 2.22045e-16 1.0      5.55112e-17
 0.0      2.77556e-17 1.0

```

Рис. 11: Пример вычисления QR-факторизации и определение составного типа факторизации для его хранения

Примеры собственной декомпозиции матрицы A (рис. 12):

```

[60]: # Симметризация матрицы A:
      Asym = A + A'

[60]: 3×3 Matrix{Float64}:
      0.652338  1.38355  0.567823
      1.38355  0.855536  0.559264
      0.567823  0.559264  1.54905

[61]: # Спектральное разложение симметризованной матрицы:
      AsymEig = eigen(Asym)

[61]: Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
      values:
      3-element Vector{Float64}:
      -0.6339734114124991
      0.9961611222370685
      2.694734318396358
      vectors:
      3×3 Matrix{Float64}:
      0.735935 -0.375553 -0.563347
      -0.676812 -0.43023 -0.59735
      -0.018032 0.820891 -0.5708

[62]: # Собственные значения:
      AsymEig.values

[62]: 3-element Vector{Float64}:
      -0.6339734114124991
      0.9961611222370685
      2.694734318396358

[63]: #Собственные векторы:
      AsymEig.vectors

[63]: 3×3 Matrix{Float64}:
      0.735935 -0.375553 -0.563347
      -0.676812 -0.43023 -0.59735
      -0.018032 0.820891 -0.5708

[64]: # Проверяем, что получится единичная матрица:
      inv(AsymEig)*Asym

[64]: 3×3 Matrix{Float64}:
      1.0      3.747e-16 -2.77556e-16
      -1.66533e-16 1.0      -1.11022e-16
      -1.66533e-16 -3.88578e-16 1.0

```

Рис. 12: Примеры собственной декомпозиции матрицы $\mathbb{A}$

Далее рассмотрим примеры работы с матрицами большой размерности и специальной структуры (рис. 13):

```
[65]: # Матрица 1000 x 1000:
n = 1000
A = randn(n,n)

[65]: 1000x1000 Matrix{Float64}:
-0.365843  0.0248239  1.94399  ... -0.571335  1.27958  -0.911608
-2.90833  -1.35017  0.752992  ... 0.824441  0.285912  1.15351
0.897415  -0.713983  -0.0876198  ... 0.945093  0.235476  -0.576157
2.4989  2.10925  0.306239  ... 0.706876  -1.00505  -1.58276
0.320019  2.08714  -0.988992  ... 1.13271  0.45312  2.58964
-0.149581  0.394433  1.53733  ... -0.656575  1.31348  -0.879211
-1.60821  -0.61793  -0.480273  ... 1.12714  -0.302029  2.08001
-0.0092169  -2.76678  -0.216163  ... 1.72591  -0.74508  0.715605
0.757835  0.726982  -0.060718  ... 0.598413  -0.388749  -0.899754
0.00037  1.81228  -0.370338  ... 0.113891  0.822378  -0.427874
0.0460105  0.469396  0.23189  ... 0.97946  0.541705  1.56422
-0.0709434  1.45583  0.311919  ... 0.138157  0.00804753  -0.392207
0.773502  1.50659  -0.408706  ... 0.715759  1.28518  0.308279
⋮
-0.0869674  -0.329045  -2.26328  ... 0.686539  0.390728  -0.506857
-0.691345  -2.64935  0.348839  ... -0.936988  0.708178  0.297679
-0.257608  -0.0651074  -0.595803  ... 1.26221  1.40962  1.81425
-0.0956575  1.90468  -0.30429  ... -0.320433  -0.138161  0.67728
0.353891  -0.399525  0.814855  ... 1.41325  0.823522  0.210694
-0.442013  0.0443326  -0.532761  ... 0.796322  0.554758  -0.0102466
1.85322  0.976253  -0.325772  ... -0.556041  -0.801917  -0.800976
-1.04291  0.819326  -0.389949  ... -0.311023  0.182417  0.957239
0.884516  0.166593  -0.456432  ... -0.281017  0.97416  0.484274
0.416266  -0.859439  -1.13121  ... 0.404443  -0.452382  -0.706339
0.470427  -0.027339  0.839858  ... -0.341844  0.715534  -0.247075
1.28134  -0.631185  -0.100903  ... -0.320209  1.01618  0.477498

[66]: # Симметризация матрицы:
Asym = A + A'

[66]: 1000x1000 Matrix{Float64}:
-0.731686  -2.88351  2.84141  ... -0.155069  1.75001  0.369729
-2.88351  -2.70033  0.0390082  ... -0.0349976  0.258573  0.522329
2.84141  0.0390082  -0.17524  ... -0.186117  1.07533  -0.677061
2.33725  2.46918  0.501657  ... 0.118643  -0.490265  -3.54919
1.8202  1.48527  -0.0151638  ... 2.35976  -1.64452  2.42915
-1.05582  -0.631029  2.12606  ... -0.396016  0.158541  -0.314692
-0.612205  0.340674  1.24882  ... 1.65993  -0.198558  0.626989
1.35578  -3.0404  0.136906  ... 2.3619  -0.484182  0.65553
-0.190427  1.11078  -0.0310555  ... 0.424813  0.286811  -0.226339
-1.08186  1.90512  0.251907  ... -0.714032  -0.725121  -0.49814
-0.669016  0.440318  1.06358  ... 2.2038  0.538697  1.50592
0.934184  0.499711  0.301354  ... -0.745057  0.653754  -1.53914
1.34407  1.75983  -1.88371  ... 1.27771  1.94142  -2.06805
⋮
-0.608362  -0.916874  -3.22758  ... 0.184663  -0.491562  -0.607401
-0.560792  -2.04353  2.55471  ... -2.14536  0.280394  -0.153336
-0.965204  0.731456  -0.53462  ... 1.82786  2.42796  1.68693
0.278592  1.26205  -0.11473  ... -0.160239  -1.18864  -1.27943
0.336421  -1.635  0.0840518  ... 1.95238  1.49298  0.40182
-0.258496  2.93988  -3.00306  ... 1.08417  0.716057  -0.79417
0.59205  2.54053  -0.891529  ... 0.402175  -1.67434  -0.947669
-0.72962  -1.72163  -0.209432  ... -2.63365  -0.934566  3.78127
0.999693  1.41278  0.005608  ... -0.0845344  0.576045  2.41729
-0.155069  -0.0349976  -0.186117  ... 0.808886  -0.794226  -1.02655
1.75001  0.258573  1.07533  ... -0.794226  1.43107  0.769107
0.369729  0.522329  -0.677061  ... -1.02655  0.769107  0.954995

[67]: # Проверка, является ли матрица симметричной:
issymmetric(Asym)

[67]: true
```

Рис. 13: Примеры работы с матрицами большой размерности и специальной структуры

Пример добавления шума в симметричную матрицу (матрица уже не будет симметричной) (рис. 14):

```
[68]: # Добавление шума:
      Asym_noisy = copy(Asym)
      Asym_noisy[1,2] += 5eps()

[68]: -2.8835071702043233

[69]: # Проверка, является ли матрица симметричной:
      issymmetric(Asym_noisy)

[69]: false
```

Рис. 14: Пример добавления шума в симметричную матрицу

В Julia можно объявить структуру матрица явно, например, используя Diagonal, Triangular, Symmetric, Hermitian, Tridiagonal и SymTridiagonal (рис. 15):

```
[70]: # Явно указываем, что матрица является симметричной:
      Asym_explicit = Symmetric(Asym_noisy)

[70]: 1000×1000 Symmetric{Float64, Matrix{Float64}}:
      -0.731686  -2.88351   2.84141   ...  -0.155069   1.75001   0.369729
      -2.88351  -2.70033   0.0390082  ...  -0.0349976  0.258573  0.522329
       2.84141   0.0390082  -0.17524   ...  -0.186117   1.07533  -0.677061
       2.33725   2.46918   0.501657   ...  0.118643  -0.490265  -3.54919
       1.8202    1.48527  -0.0151638  ...  2.35976   -1.64452   2.42915
      -1.05582  -0.631029   2.12606   ...  -0.396016   0.158541  -0.314692
      -0.612205  0.340674   1.24882   ...  1.65993   -0.198558  0.626989
       1.35578  -3.0404    0.136906   ...  2.3619   -0.484182  0.65553
      -0.190427  1.11078   -0.0310555  ...  0.424813   0.286811  -0.226339
      -1.08186   1.90512   0.251907   ...  -0.714032  -0.725121  -0.49814
      -0.669016  0.440318   1.86358   ...  2.2038    0.538697  1.50592
       0.934184  0.499711   0.301354   ...  -0.745057  0.653754  -1.53914
       1.34407   1.75983  -1.88371   ...  1.27771   1.94142  -2.06805
       ⋮
      -0.608362  -0.916874  -3.22758   ...  0.184663  -0.491562  -0.607401
      -0.560792  -2.04353   2.55471   ...  -2.14536   0.280394  -0.153336
      -0.965204  0.731456  -0.53462   ...  1.82786   2.42796   1.68693
       0.278592   1.26205  -0.11473   ...  -0.160239  -1.18864  -1.27943
       0.336421  -1.635    0.0840518  ...  1.95238   1.49298   0.40182
      -0.258496  2.93988   -3.00306   ...  1.08417   0.716057  -0.79417
       0.59205    2.54053  -0.891529   ...  0.402175  -1.67434  -0.947669
      -0.72962  -1.72163  -0.209432   ...  -2.63365  -0.934566  3.78127
       0.999693   1.41278   0.805608   ...  -0.0845344  0.576045  2.41729
      -0.155069  -0.0349976  -0.186117   ...  0.808886  -0.794226  -1.02655
       1.75001    0.258573   1.07533   ...  -0.794226  1.43107   0.769107
       0.369729  0.522329  -0.677061  ...  -1.02655   0.769107  0.954995
```

Рис. 15: Пример явного объявления структуры матрицы

Далее для оценки эффективности выполнения операций над матрицами большой размерности и специальной структуры воспользуемся пакетом BenchmarkTools (рис. 16):

```
[89]: using BenchmarkTools
      # Оценка эффективности выполнения операции по нахождению
      # собственных значений симметризованной матрицы:
      @btime eigvals(Asym);

      207.412 ms (18 allocations: 7.98 MiB)

[87]: # Оценка эффективности выполнения операции по нахождению
      # собственных значений зашумлённой матрицы:
      @btime eigvals(Asym_noisy);

      1.017 s (27 allocations: 7.93 MiB)

[88]: # Оценка эффективности выполнения операции по нахождению
      # собственных значений зашумлённой матрицы,
      # для которой явно указано, что она симметричная:
      @btime eigvals(Asym_explicit);

      196.689 ms (18 allocations: 7.98 MiB)
```

Рис. 16: Использование пакета *BenchmarkTools*

Далее рассмотрим примеры работы с разреженными матрицами большой размерности.

Использование типов *Tridiagonal* и *SymTridiagonal* для хранения трёхдиагональных матриц позволяет работать с потенциально очень большими трёхдиагональными матрицами (рис. 17):

[illegible]

В примере показано, как можно решить систему линейных уравнений с рациональными элементами без преобразования в типы элементов с плавающей запятой (для избежания проблемы с переполнением используем `BigInt`) (рис. 18):

```

[92]: # Матрица с рациональными элементами:
Arational = Matrix{Rational{BigInt}}(rand(1:10, 3, 3))/10

[92]: 3×3 Matrix{Rational{BigInt}}:
 2//5  2//5  4//5
 1//10 2//5  1//5
 3//5  1//2  7//10

[93]: # Единичный вектор:
x = fill(1, 3)

[93]: 3-element Vector{Int64}:
 1
 1
 1

[94]: # Задаём вектор b:
b = Arational*x

[94]: 3-element Vector{Rational{BigInt}}:
 8//5
 7//10
 9//5

[97]: # Решение исходного уравнения получаем с помощью функции \
# (убеждаемся, что x - единичный вектор):
Arational\b

[97]: 3-element Vector{Rational{BigInt}}:
 1
 1
 1

[98]: # LU-разложение:
lu(Arational)

[98]: LU{Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int64}}
L factor:
3×3 Matrix{Rational{BigInt}}:
 1  0  0
 1//6 1  0
 2//3 4//19 1
U factor:
3×3 Matrix{Rational{BigInt}}:
 3//5  1//2  7//10
 0  19//60  1//12
 0  0  6//19

```

Рис. 18: Решение системы линейных уравнений с рациональными элементами без преобразования в типы элементов с плавающей запятой

2.7 Самостоятельная работа

Выполнение задания “Произведение векторов” (рис. 19):

```
[100]: # Самостоятельная работа

[101]: using LinearAlgebra

# 1. Задайте вектор v
v = [1, 2, 3, 4, 5]

# Умножьте вектор v скалярно сам на себя
dot_v = dot(v, v)

[101]: 55

[103]: # Матричное (внешнее) произведение
outer_v = v * v'

[103]: 5×5 Matrix{Int64}:
 1  2  3  4  5
 2  4  6  8 10
 3  6  9 12 15
 4  8 12 16 20
 5 10 15 20 25
```

Рис. 19: Решение задания “Произведение векторов”

Выполнение задания “Системы линейных уравнений” (рис. 20 - рис. 21):

```
[104]: using LinearAlgebra

# a)  $x + y = 2, x - y = 3$ 
A1 = [1 1; 1 -1]
b1 = [2, 3]
x1 = A1 \ b1
println("a) Решение: x = $(x1[1]), y = $(x1[2])")

# b)  $x + y = 2, 2x + 2y = 4$ 
A2 = [1 1; 2 2]
b2 = [2, 4]
try
    x2 = A2 \ b2
    println("b) Решение: x = $(x2[1]), y = $(x2[2])")
catch e
    println("b) Система имеет бесконечно много решений или несовместна")
end

# c)  $x + y = 2, 2x + 2y = 5$ 
A3 = [1 1; 2 2]
b3 = [2, 5]
try
    x3 = A3 \ b3
    println("c) Решение: x = $(x3[1]), y = $(x3[2])")
catch e
    println("c) Система несовместна")
end

# d) Переопределенная система
A4 = [1 1; 2 2; 3 3]
b4 = [1, 2, 3]
x4 = A4 \ b4
println("d) Решение методом наименьших квадратов: x = $(x4[1]), y = $(x4[2])")

# e)  $x + y = 2, 2x + y = 1, x - y = 3$ 
A5 = [1 1; 2 1; 1 -1]
b5 = [2, 1, 3]
x5 = A5 \ b5
println("e) Решение методом наименьших квадратов: x = $(x5[1]), y = $(x5[2])")

# f)  $x + y = 2, 2x + y = 1, 3x + 2y = 3$ 
A6 = [1 1; 2 1; 3 2]
b6 = [2, 1, 3]
x6 = A6 \ b6
println("f) Решение методом наименьших квадратов: x = $(x6[1]), y = $(x6[2])")

a) Решение: x = 2.5, y = -0.5
b) Система имеет бесконечно много решений или несовместна
c) Система несовместна
d) Решение методом наименьших квадратов: x = 0.4999999999999999, y = 0.5
e) Решение методом наименьших квадратов: x = 1.5000000000000007, y = -0.9999999999999997
f) Решение методом наименьших квадратов: x = -0.9999999999999999, y = 2.9999999999999982
```

Рис. 20: Решение задания “Системы линейных уравнений”

```

[110]: using LinearAlgebra

# a)  $x + y + z = 2, x - y - 2z = 3$ 
A7 = [1 1 1; 1 -1 -2]
b7 = [2, 3]
x7 = A7 \ b7
println("a) Решение: x = $(x7[1]), y = $(x7[2]), z = $(x7[3])")

# b)  $x + y + z = 2, 2x + 2y - 3z = 4, 3x + y + z = 1$ 
A8 = [1 1 1; 2 2 -3; 3 1 1]
b8 = [2, 4, 1]
x8 = A8 \ b8
println("b) Решение: x = $(x8[1]), y = $(x8[2]), z = $(x8[3])")

# c)  $x + y + z = 1, x + y + 2z = 0, 2x + 2y + 3z = 1$ 
A9 = [1 1 1; 1 1 2; 2 2 3]
b9 = [1, 0, 1]
try
    x9 = A9 \ b9
    println("c) Решение: x = $(x9[1]), y = $(x9[2]), z = $(x9[3])")
catch e
    println("c) Система вырожденная или несовместная")
end

# d)  $x + y + z = 1, x + y + 2z = 0, 2x + 2y + 3z = 0$ 
A10 = [1 1 1; 1 1 2; 2 2 3]
b10 = [1, 0, 0]
try
    x10 = A10 \ b10
    println("d) Решение: x = $(x10[1]), y = $(x10[2]), z = $(x10[3])")
catch e
    println("d) Система вырожденная или несовместная")
end

a) Решение: x = 2.2142857142857144, y = 0.35714285714285704, z = -0.5714285714285712
b) Решение: x = -0.5, y = 2.5, z = 0.0
c) Система вырожденная или несовместная
d) Система вырожденная или несовместная

```

Рис. 21: Решение задания “Системы линейных уравнений”

Выполнение задания “Операции с матрицами” (рис. 22 - рис. 26):

```
[111]: A = [1 -2; -2 1]

eigen_A = eigen(A)
diagMatrix = Diagonal(eigen_A.values)

[111]: 2×2 Diagonal{Float64, Vector{Float64}}:
-1.0  .
.    3.0

[112]: B = [1 -2; -2 3]

eigen_B = eigen(B)
diagMatrix = Diagonal(eigen_B.values)

[112]: 2×2 Diagonal{Float64, Vector{Float64}}:
-0.236068  .
.          4.23607

[113]: C = [1 -2; -2 3]

eigen_C = eigen(C)
diagMatrix = Diagonal(eigen_C.values)

[113]: 2×2 Diagonal{Float64, Vector{Float64}}:
-0.236068  .
.          4.23607
```

Рис. 22: Решение задания “Операции с матрицами”

```
[114]: A = [1 -2;
           -2 1]

# Собственные значения и векторы
eigen_decomp = eigen(A)
P = eigen_decomp.vectors
D = Diagonal(eigen_decomp.values)

D_10 = D.^10

A_10 = P * D_10 * inv(P)

println("Матрица A^10: ")
println(A_10)

Матрица A^10:
[29525.0 -29524.0; -29524.0 29525.0]
```

Рис. 23: Решение задания “Операции с матрицами”

```
[115]: A = [5 -2;
           -2 5]

# Собственные значения и векторы
eigen_decomp = eigen(A)
eigen_values = eigen_decomp.values
eigen_vectors = eigen_decomp.vectors

if all(eigen_values .>= 0)
    sqrt_D = Diagonal(sqrt.(eigen_values))
    sqrt_A = eigen_vectors * sqrt_D * inv(eigen_vectors)

    println("Матрица A: ")
    println(A)

    println("Матрица A (квадратный корень): ")
    println(sqrt_A)

    println("Матрица A (квадратный корень)^2: ")
    println(sqrt_A * sqrt_A)
else
    println("Матрица A имеет отрицательные собственные значения (квадратный корень не определён)")
end

Матрица A:
[5 -2; -2 5]
Матрица A (квадратный корень):
[2.188901059316734 -0.45685025174785676; -0.45685025174785676 2.188901059316734]
Матрица A (квадратный корень)^2:
[5.000000000000002 -2.000000000000001; -2.000000000000001 5.000000000000002]
```

Рис. 24: Решение задания “Операции с матрицами”

```
[117]: A = [1 -2;
           -2 1]

# Собственные значения и векторы
eigen_decomp = eigen(A)
eigen_values = eigen_decomp.values
eigen_vectors = eigen_decomp.vectors

complex_eigen_values = Complex.(eigen_values)
cube_root_D = Diagonal(complex_eigen_values .^ (1/3))

cube_root_A = eigen_vectors * cube_root_D * inv(eigen_vectors)

D_10 = D.^10
A_10 = P * D_10 * inv(P)

println("Матрица A: ")
println(A)
println("\n")

println("Матрица A (кубический корень): ")
println(cube_root_A)
println("\n")

println("Проверка матрицы A (кубический корень): ")
println(cube_root_A * cube_root_A * cube_root_A)
println("\n")

Матрица A:
[1 -2; -2 1]

Матрица A (кубический корень):
ComplexF64[0.971124785153704 + 0.4330127018922193im -0.4711247851537041 + 0.4330127018922193im; -0.4711247851537041 + 0.4330127018922193im 0.971124785153704 + 0.4330127018922193im]

Проверка матрицы A (кубический корень):
ComplexF64[0.9999999999999993 + 0.0im -1.9999999999999991 + 0.0im; -1.9999999999999991 + 0.0im 0.9999999999999993 + 0.0im]
```

Рис. 25: Решение задания “Операции с матрицами”

```

3) Найдите собственные значения матрицы A. Создайте диагональную матрицу из собственных значений матрицы A. Создайте нижнедиагональную матрицу из матрицы A. Оцените эффективность выполненных операций.

[[110]: # Исходная матрица A
A = [
    140  97  74 168 131;
    97 186  89 131 36;
    74  89 152 144  71;
    168 131 144  54 142;
    131 36  71 142 36
]

# 1. Нахождение собственных значений и векторов
@time eigen_decomp = eigen(A)
eigenvalues = eigen_decomp.values
eigenvectors = eigen_decomp.vectors

println("Собственные значения матрицы A:")
println(eigenvalues)

# 2. Создание диагональной матрицы из собственных значений
# Прямое создание переменной и вывод без использования @time
diag_matrix = Diagonal(eigenvalues)

println("\nДиагональная матрица из собственных значений:")
println(Matrix{typeof(diag_matrix)}{1,000} # Преобразуем в стандартный массив для вывода

# 3. Создание нижнедиагональной матрицы из A
lower_triangular = lowerTriangular(A)

println("\nНижнедиагональная матрица из A:")
println(Matrix{typeof(lower_triangular)}{1,000}

# 4. Оценка эффективности
println("\nЭффективность выполнения операций:")
@time eigen(A)
@time Diagonal(eigenvalues)
@time lowerTriangular(A)

5.433 μs (11 allocations: 3.00 KiB)
Собственные значения матрицы A:
[-1.0, 3.0]

Диагональная матрица из собственных значений:
[-1.0 0.0; 0.0 3.0]

Нижнедиагональная матрица из A:
[140 0 0 0; 97 186 0 0; 74 89 152 0; 168 131 144 54; 131 36 71 142 36]

Эффективность выполнения операций:
5.383 μs (11 allocations: 3.00 KiB)
193.209 ns (1 allocation: 16 bytes)
177.997 ns (1 allocation: 16 bytes)

[[110]: 5×5 LowerTriangular{Int64, Matrix{Int64}}:
 140  -  -  -  -
   97 186  -  -  -
   74  89 152  -  -
  168 131 144  54  -
   131 36  71 142 36

```

Рис. 26: Решение задания “Операции с матрицами”

Выполнение задания “Линейные модели экономики” (рис. 27):


```
[126]: A1 = [1 2; 3 4]
A2 = (1/2) * A1
A3 = (1/10) * A1
A4 = [0.1 0.2 0.3; 0 0.1 0.2; 0 0.1 0.3]

function checkProductivityViaInverse(A)
    E = I(size(A, 1))
    B = E - A
    try
        B_Inv = inv(B)
        all(B_Inv .>= 0)
    catch
        false
    end
end

function checkProductivityViaSpectrum(A)
    eigen_values = eigvals(A)
    all(abs.(eigen_values) .< 1)
end

matrices = [A1, A2, A3, A4]
for (i, A) in enumerate(matrices)
    println("Матрица A$i: ")
    println(A)
    println("Via Inverse: ", checkProductivityViaInverse(A))
    println("Via Spectrum: ", checkProductivityViaSpectrum(A))
    println("-"^30)
end

Матрица A1:
[1.0 2.0; 3.0 4.0]
Via Inverse: false
Via Spectrum: false
-----
Матрица A2:
[0.5 1.0; 1.5 2.0]
Via Inverse: false
Via Spectrum: false
-----
Матрица A3:
[0.1 0.2; 0.30000000000000004 0.4]
Via Inverse: false
Via Spectrum: true
-----
Матрица A4:
[0.1 0.2 0.3; 0.0 0.1 0.2; 0.0 0.1 0.3]
Via Inverse: false
Via Spectrum: true
-----
```

Рис. 27: Решение задания “Линейные модели экономики”

3 Вывод

В ходе выполнения лабораторной работы были изучены возможности специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

4 Список литературы. Библиография

[1] Julia Documentation: <https://docs.julialang.org/en/v1/>